

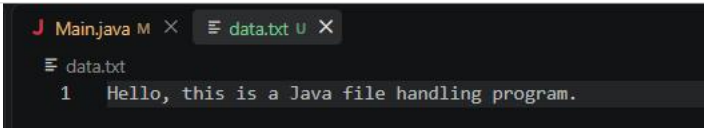


Symbiosis Institute of Technology, Pune

CSE- Academic Year 2026-27

Java Programming – Lab Batch 2025-29

Lab Assignment No.:	
Name of Student	Akshit Gawade
PRN No.	25070122029
Batch	2025-2029
Class	A1
Academic Year & Semester	Second Year Semester III
Date of Submission	
Title of Assignment:	Write a Java program to perform reading from and writing to a file using file handling operations.
Problem Statement & Learning Objective	Problem Statement Write a Java program to perform reading from and writing to a file using file handling operations. Learning Objectives To perform file reading and writing operations using Java file handling classes.
Theory	Java provides file handling through the java.io package. Classes such as File, FileReader, FileWriter, BufferedReader, and BufferedWriter are used to perform input and output operations. File handling enables permanent storage and retrieval of application data. Proper exception handling is necessary to manage file-related errors. These concepts are essential for data persistence.
Program Code	<pre>import java.io.*; public class Main { public static void main(String[] args) { try { FileWriter fw = new FileWriter("Exp13/data.txt"); fw.write("Hello, this is a Java file handling</pre>

	<pre> program."); fw.close(); FileReader fr = new FileReader("Exp13/data.txt"); BufferedReader br = new BufferedReader(fr); String line; while ((line = br.readLine()) != null) { System.out.println(line); } br.close(); fr.close(); } catch (IOException e) { System.out.println("File error: " + e.getMessage()); } } </pre>
Output Screenshots	
Exercise Qns and Code	1. Create a program to write and read employee details from a file. <pre> import java.io.*; public class Ex1 { public static void main (String[] args) { try { FileWriter fw = new FileWriter("Exercise/employee.txt"); fw.write("Employee ID: 101\n"); fw.write("Name: Rahul\n"); fw.write("Salary: 50000\n"); fw.close(); FileReader fr = new FileReader("Exercise/employee.txt"); BufferedReader br = new BufferedReader(fr); String line; while ((line = br.readLine()) != null) { System.out.println(line); } } } } </pre>

	<pre> } br.close(); fr.close(); } catch (IOException e) { System.out.println("File error: " + e.getMessage()); } } </pre> 2. Create a Java program to write patient details (patient ID, name, age, and diagnosis) to a file and read the details from the file for display. <pre> import java.io.*; public class Exp2 { public static void main (String[] args) { try { FileWriter fw = new FileWriter("Exercise/patient.txt"); fw.write("Patient ID: 101\n"); fw.write("Name: Amit\n"); fw.write("Age: 25\n"); fw.write("Diagnosis: Fever\n"); fw.close(); FileReader fr = new FileReader("Exercise/patient.txt"); BufferedReader br = new BufferedReader(fr); String line; while ((line = br.readLine()) != null) { System.out.println(line); } br.close(); fr.close(); } catch (IOException e) { System.out.println("File error: " + e.getMessage()); } } } } </pre>
Output Screenshots	Ex1:

```
Problems Output Debug Console Terminal Ports
PS C:\Users\Nilima\Documents\JAVA PROJECTS\LAB EXP 13> & 'c:\m\jdk-24.0.2.12-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInUsers\Nilima\AppData\Roaming\Code\User\workspaceStorage\39af021hat.java\jdt_ws\LAB EXP 13_cc80a22a\bin' 'Ex1'
Employee ID: 101
Name: Rahul
Salary: 50000
PS C:\Users\Nilima\Documents\JAVA PROJECTS\LAB EXP 13>
```

```
employee.txt U X patient.txt U X
Exercise > employee.txt
1 Employee ID: 101
2 Name: Rahul
3 Salary: 50000
4
```

Ex2:

```
Problems Output Debug Console Terminal Ports
PS C:\Users\Nilima\Documents\JAVA PROJECTS\LAB EXP 13> & 'c:\m\jdk-24.0.2.12-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInUsers\Nilima\AppData\Roaming\Code\User\workspaceStorage\39af021hat.java\jdt_ws\LAB EXP 13_cc80a22a\bin' 'Exp2'
Patient ID: 101
Name: Amit
Age: 25
Diagnosis: Fever
PS C:\Users\Nilima\Documents\JAVA PROJECTS\LAB EXP 13>
```

```
employee.txt U X patient.txt U X
Exercise > patient.txt
1 Patient ID: 101
2 Name: Amit
3 Age: 25
4 Diagnosis: Fever
5
```

Conclusion

This experiment explores persistent data storage using Java's standard file stream classes. By implementing `BufferedWriter` and `BufferedReader`, it demonstrates how buffered character streams optimize file operations to efficiently write, store, and process structured records across application restarts.